

Name of the Student	Yoga Madha A
Reg. No	192425058
GitHub Link	https://github.com/YOGAMADHA/MLA0405-DEEP-LEARNING

SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 2

Industry Problem Solving Task

COURSE NAME: Deep Learning
SLOT :

COURSE CODE: MLA04

COURSE OUTCOME COVERED

CO 4: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems. BL-6

Course Outcome Weightage: 10%
Assessment Tool Weightage: 40%

Duration: 90 minutes
Mark : 25

Code of Conduct:

I Yoga Madha A-192425058 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Name of the student:

Criteria	Understanding of Industry Problem (2)	Application of Engineering Concepts (2.5)	Solution Design (2.5)	Use of Tools (2)	Analysis (1)	Total (10)
Weightage	20 %	25%	25%	20 %	10 %	100%
Q1						
Q2						
Q3						
Q4						
Q5						
Total						

QUESTIONS: 05

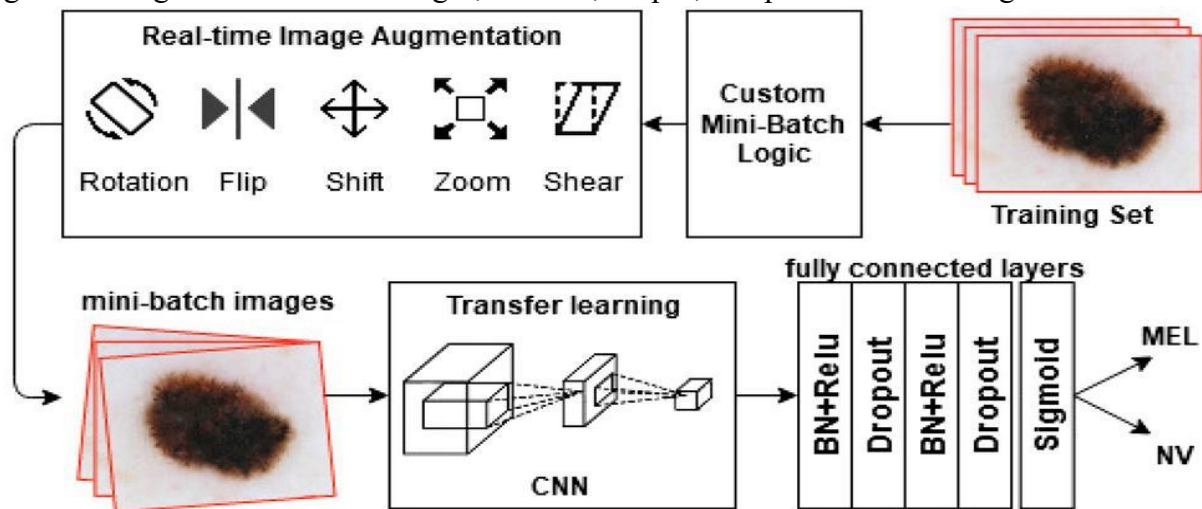
Total Marks:25

Q1. Transfer Learning

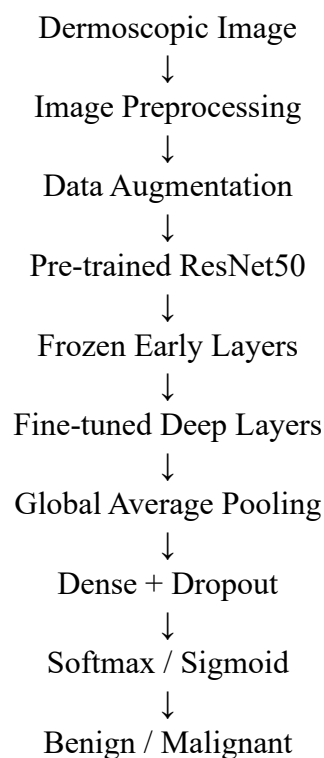
A healthcare startup aims to develop an AI-based system for detecting skin cancer from dermoscopic images. Since only a limited number of labeled images are available, design a transfer learning strategy using a pre-trained CNN model. Justify your choice of model, identify which layers should be frozen or fine-tuned, and explain how your design improves classification performance.

Answer:

A healthcare startup can use **transfer learning with a pre-trained ResNet50 CNN** to detect skin cancer from dermoscopic images. Since the available labeled dataset is small, training a CNN from scratch may cause overfitting. ResNet50 is suitable because it has already learned general image features such as edges, textures, shapes, and patterns from a large dataset.



Proposed Architecture:



The **early layers of ResNet50 are frozen** because they learn general features such as edges and textures. The **last few deep layers are fine-tuned** using a low learning rate so that they can learn skin-lesion-specific features such as irregular borders, pigmentation, texture, and shape.

Data augmentation techniques such as **rotation, flipping, zooming, and shifting** increase the diversity of the limited training dataset. **Dropout** is added to reduce overfitting.

The model can be implemented using **Python, TensorFlow/Keras, ResNet50, OpenCV/Pillow, NumPy, and scikit-learn**. Performance can be evaluated using **accuracy, precision, recall, F1-score, AUC, and a confusion matrix**. High recall is especially important because incorrectly classifying a malignant lesion as benign can be a serious error.

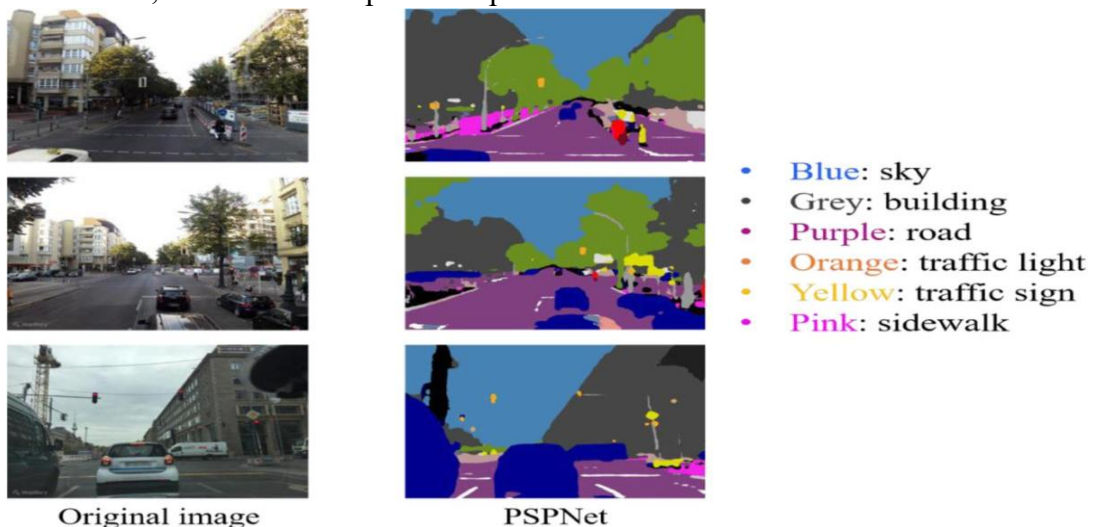
Thus, transfer learning provides a **faster, practical, and accurate solution** for skin cancer classification with limited labeled data.

Q2. DenseNet and PixelNet

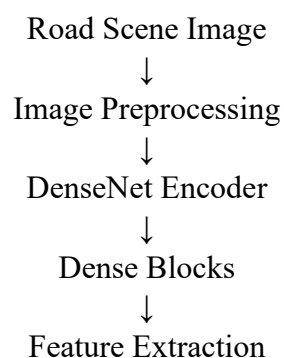
A smart city project requires an automated road-scene analysis system that accurately identifies roads, pedestrians, vehicles, and traffic signs at the pixel level. Design a deep learning architecture using DenseNet and PixelNet to solve this problem. Explain how the proposed architecture improves feature reuse, segmentation accuracy, and computational efficiency.

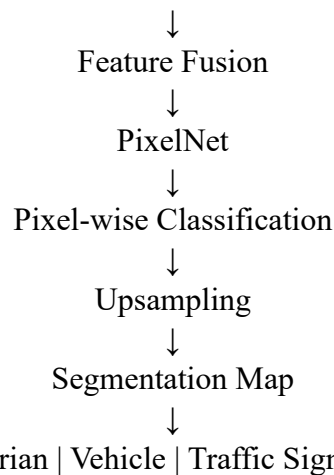
Answer:

A smart city requires an automated **road-scene analysis system** that can identify roads, pedestrians, vehicles, and traffic signs at the **pixel level**. A combination of **DenseNet and PixelNet** can be used for accurate semantic segmentation. DenseNet acts as the feature extraction network, while PixelNet performs pixel-level classification.



Proposed Architecture:





DenseNet for Feature Reuse:DenseNet connects each layer with the following layers, allowing previously extracted features to be reused. This improves information and gradient flow and reduces the need to relearn similar features.

PixelNet for Pixel-level Segmentation:PixelNet analyzes individual pixels and assigns a class label to each pixel. Therefore, different parts of the road scene can be accurately separated into road, pedestrian, vehicle, traffic sign, and background.

Improved Segmentation Accuracy:DenseNet provides detailed low-level and high-level features, while PixelNet performs pixel-wise prediction. This helps identify both large objects such as roads and small objects such as pedestrians and traffic signs.

Computational Efficiency:

Dense feature reuse reduces redundant feature learning and can reduce the number of parameters. Feature fusion and efficient upsampling also help produce detailed segmentation without unnecessarily increasing computational cost.

Tools: Python, TensorFlow/Keras or PyTorch, OpenCV, NumPy, and Matplotlib can be used for implementation and visualization.

Analysis: The system can be evaluated using **Pixel Accuracy**, **Mean IoU (mIoU)**, **Precision**, **Recall**, and **F1-score**. A high mIoU indicates that the predicted segmentation closely matches the actual road-scene regions.

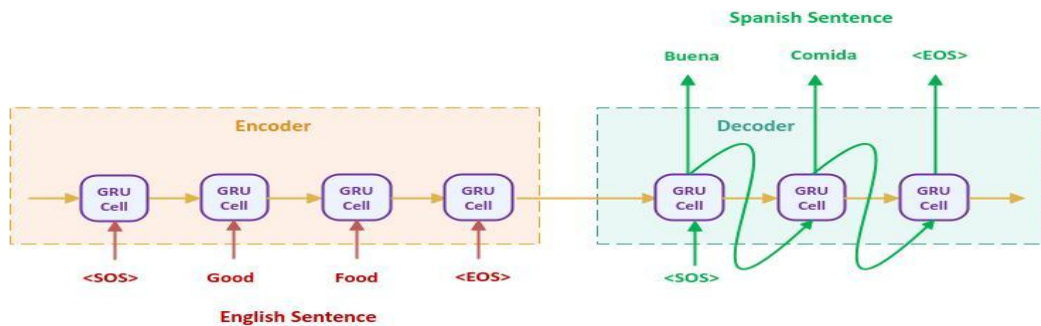
Conclusion: The combination of **DenseNet for efficient feature extraction** and **PixelNet for pixel-level classification** provides an accurate and efficient solution for automated smart-city road-scene segmentation.

Q3. Bidirectional RNN and Encoder–Decoder

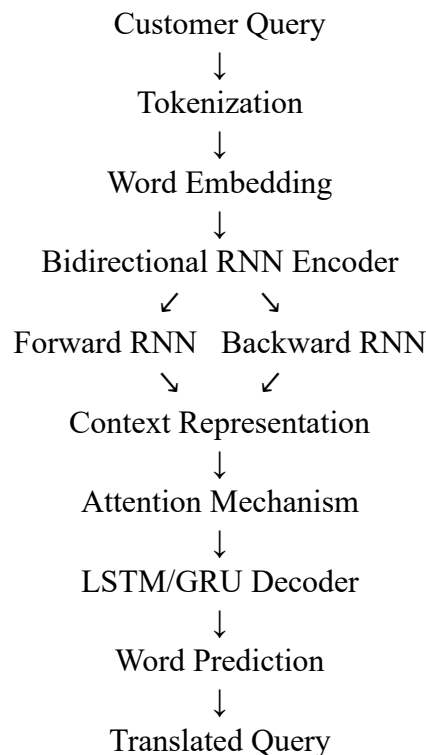
A multinational customer service company wants to build a multilingual chatbot capable of translating customer queries while preserving contextual meaning. Design an Encoder–Decoder sequence-to-sequence model using Bidirectional RNNs. Illustrate the architecture and justify how bidirectional processing improves translation accuracy.

Answer:

A multinational customer service company can develop a **multilingual chatbot using a Bidirectional RNN-based Encoder–Decoder model**. The system translates customer queries from one language to another while preserving the meaning and context of the sentence. The **Bidirectional RNN encoder** reads the input sentence in both forward and backward directions, providing better contextual information to the decoder.



Proposed Architecture:



The **Encoder** converts the input sentence into contextual feature representations. The forward RNN processes the sentence from beginning to end, while the backward RNN processes it from end to beginning.

For an input sentence:

“I need help with my account.”

the forward network reads:

I → need → help → with → my → account

and the backward network reads:

account → my → with → help → need → I

The two representations are combined to provide information from **both past and future context**.

The **Decoder** uses this contextual information to generate the translated sentence one word at a time. An **attention mechanism** can help the decoder focus on the most relevant input words during translation.

Bidirectional processing improves translation because the meaning of a word can depend on words appearing **before and after it**. This helps the chatbot handle grammar, word relationships, and contextual meaning more accurately.

Tools: Python, TensorFlow/Keras or PyTorch, NumPy, and NLP libraries such as NLTK or spaCy can be used.

Analysis: Translation performance can be evaluated using **BLEU score, translation accuracy, loss, and validation performance.**

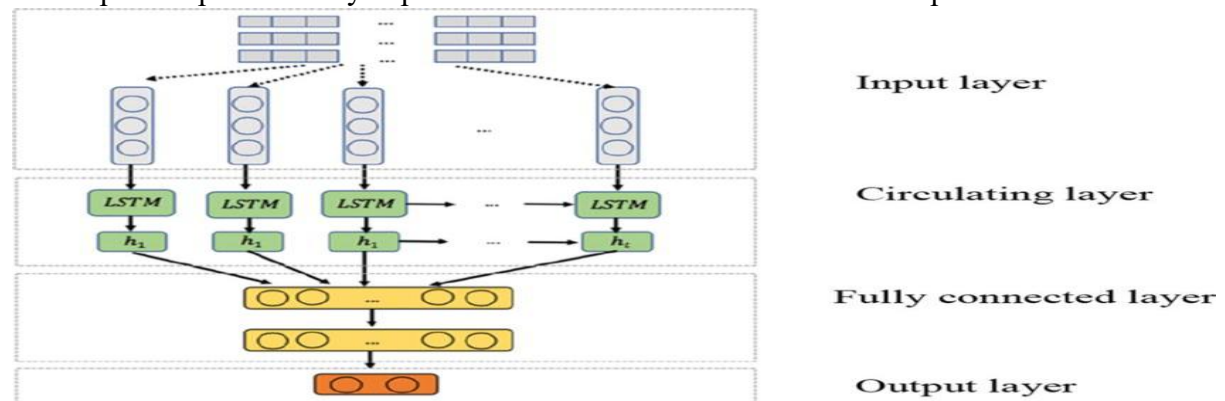
Conclusion: A **Bidirectional RNN Encoder–Decoder with attention** provides better contextual understanding than a one-directional RNN, making it suitable for accurate multilingual customer-service translation.

Q4. BPTT and LSTM

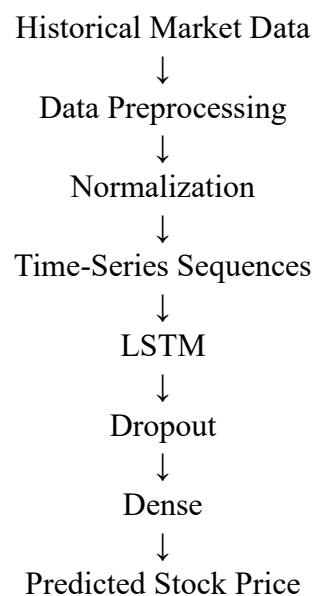
A financial analytics company is developing a model to forecast stock prices using historical market data. Create an LSTM-based prediction system and explain how Backpropagation Through Time (BPTT) is used to train the network. Justify why LSTM is more suitable than a traditional RNN for this application.

Answer:

A financial analytics company can use an **LSTM-based prediction system** to forecast stock prices from historical market data. LSTM is suitable because stock prices form a time sequence and important patterns may depend on information from earlier time steps.



Proposed Architecture:



The input data can include **opening price, closing price, highest price, lowest price, trading volume, and technical indicators.** The data is arranged into time windows, such as previous 30 days, to predict the next day's price.

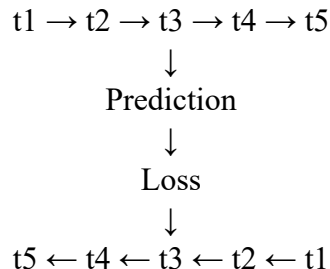
LSTM Structure

LSTM uses three main gates:

- **Forget Gate** – removes irrelevant past information.
- **Input Gate** – stores important new information.
- **Output Gate** – controls the information passed to the next time step.

Backpropagation Through Time (BPTT)

BPTT trains the LSTM by unfolding the network across time:



The model first produces a prediction and calculates the error using a loss function such as **Mean Squared Error (MSE)**. The error is then propagated backward through the time steps. The gradients are used to update the LSTM weights using an optimizer such as **Adam**.

Why LSTM is Better Than Traditional RNN

Traditional RNNs can suffer from the **vanishing-gradient problem**, making it difficult to remember information over long sequences. LSTM uses its memory cell and gates to retain important information for longer periods.

Therefore, LSTM is better suited for stock-price forecasting because it can learn **long-term temporal dependencies and historical patterns**.

Tools and Analysis

Tools: Python, TensorFlow/Keras, NumPy, Pandas, Matplotlib, and scikit-learn.

Performance can be evaluated using **MAE, MSE, RMSE, and MAPE** by comparing predicted prices with actual prices.

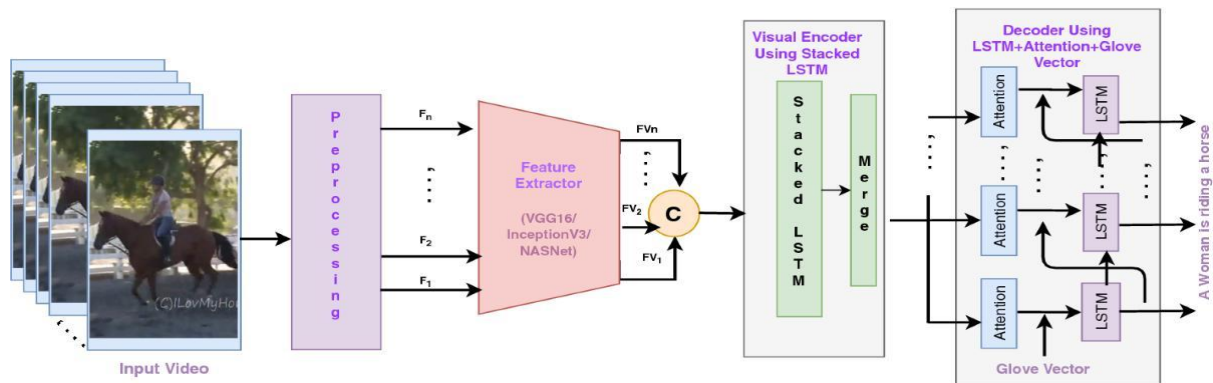
Conclusion: An LSTM trained using BPTT provides an effective sequence-learning approach for stock-price forecasting because its gated memory mechanism can retain important historical information better than a traditional RNN.

Q5. Integrated Deep Learning Solution

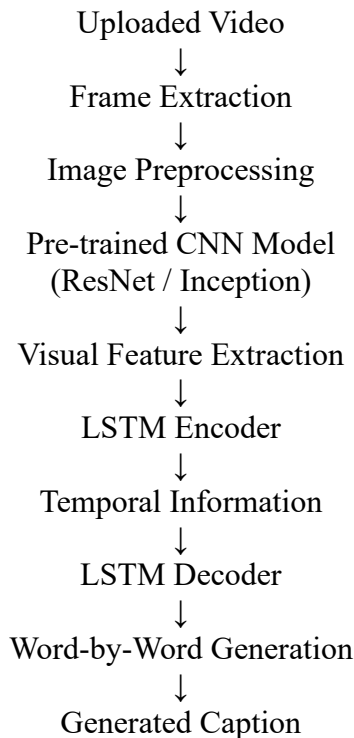
A media streaming company plans to automatically generate captions for uploaded videos to improve accessibility. Design an integrated deep learning solution that combines Transfer Learning for visual feature extraction with an LSTM-based Encoder–Decoder architecture for caption generation. Explain the role of each component and justify how the complete system produces accurate captions.

Answer:

A media streaming company can develop an **automatic video captioning system** by combining **Transfer Learning for visual feature extraction** with an **LSTM Encoder–Decoder** for generating captions. The CNN identifies important visual information from video frames, while the LSTM learns the sequence of events and converts them into meaningful sentences.



Proposed Architecture



Role of Transfer Learning

A pre-trained CNN such as **ResNet50** is used to extract visual features from individual video frames. Since the CNN has already learned features such as **edges, shapes, textures, and objects**, it does not need to be trained from scratch.

For example:

Video Frames → CNN → Person, Car, Ball, Road, etc.

This reduces training time and works effectively when the captioning dataset is limited.

Role of LSTM Encoder

The LSTM encoder receives visual features from consecutive frames:

Frame 1 → Frame 2 → Frame 3 → Frame 4

It learns the **temporal relationship** between the frames and understands how the scene changes over time.

Role of LSTM Decoder

The decoder converts the learned visual and temporal information into words.

For example:

→ A → man → is → playing → football →

At every step, the decoder predicts the next most probable word based on the previous words and encoded video information.

How the Complete System Produces Accurate Captions

- **Transfer Learning CNN** extracts meaningful visual features.
- **Frame extraction** provides information about different moments in the video.
- **LSTM Encoder** learns relationships between consecutive frames.
- **LSTM Decoder** generates captions sequentially.
- **Attention mechanism**, if included, can focus on important frames or features.
- **Softmax** selects the most probable next word.

Tools

Python, TensorFlow/Keras or PyTorch, OpenCV, NumPy, and Matplotlib can be used to implement and evaluate the system.

Analysis

Caption quality can be evaluated using **BLEU, METEOR, ROUGE, CIDEr**, and human evaluation. These metrics compare generated captions with reference captions.

Conclusion

The combination of **Transfer Learning and LSTM Encoder–Decoder** provides an effective video-captioning solution. The CNN understands **what is present in the video**, while the LSTM understands **how events occur over time** and converts them into meaningful natural-language captions. This makes the system useful for **accessibility, video search, and automatic content description**.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	20%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis